

Proyecto: VALGREEN — Sistema de Gestión de Repostería

Documento: Análisis Sistema

Generado el: 2026-09-23 16:25

PROMPT 1 — Análisis del sistema antes de probar

Nombre del proyecto: VALGREEN — Sistema de Gestión de Repostería

Descripción general: Sistema web desarrollado en PHP/Laravel y MySQL para la gestión integral de una repostería. Permite administrar productos, controlar stock, registrar clientes, procesar ventas directas, gestionar pedidos personalizados (incluyendo tortas personalizadas) y administrar usuarios con roles diferenciados.

Problema que resuelve: Centraliza y automatiza la gestión operativa de una repostería, evitando el control manual de inventarios, pedidos y ventas. Facilita el seguimiento de pedidos, el control de stock y la trazabilidad de transacciones.

Usuarios del sistema:

1. **Administrador:** Acceso completo al sistema (gestión de productos, stock, usuarios y consulta de todos los módulos).
2. **Vendedor:** Puede consultar stock, registrar ventas, gestionar pedidos y clientes.

Módulos identificados:

1. Autenticación y sesiones
2. Dashboard (Administrador / Vendedor)
3. Gestión de Productos y Categorías
4. Gestión de Stock y Movimientos
5. Gestión de Usuarios y Roles
6. Gestión de Clientes
7. Gestión de Ventas Directas
8. Gestión de Pedidos (incluye Torta Personalizada)
9. Gestión de Estados de Pedido

Funcionalidades principales:

- Inicio/cierre de sesión con control de estado activo del usuario
- Dashboard con métricas (ventas del día, pedidos activos/pendientes, stock bajo, etc.)
- CRUD de productos con imagen, categoría y precio
- Registro de movimientos de stock (Entrada / Salida / Ajuste) con auditoría
- CRUD de usuarios (con cambio de estado activo/inactivo)

- Registro de clientes con carnet único
- Registro de ventas directas con descuento automático de stock (transaccional)
- Registro de pedidos con anticipo, productos y/o torta personalizada
- Actualización de estado de pedidos y registro de pago final
- Visualización detallada de pedidos

Tecnologías: PHP, Laravel 11+, MySQL, Bootstrap 5, Blade Templates, Eloquent ORM

Tipo de aplicación: Web responsive

ANÁLISIS ORIENTADO A PRUEBAS

1. Módulos que deben ser probados

Módulo	Criticidad	Justificación
Autenticación	CRÍTICA	Punto de acceso único al sistema
Roles y Permisos	CRÍTICA	Asegura separación de funciones Administrador/Vendedor
Ventas	CRÍTICA	Afecta stock y transacciones económicas
Pedidos	CRÍTICA	Proceso central del negocio
Stock	ALTA	Integridad del inventario
Productos	ALTA	Catálogo base del sistema
Usuarios	ALTA	Gestión de accesos
Clientes	MEDIA	Datos base para pedidos
Dashboard	MEDIA	Visualización de métricas

2. Funcionalidades críticas

1. **Login correcto con usuario activo** — bloqueo de usuarios inactivos
2. **Venta con transaccionalidad** — descuento de stock atómico + validación stock suficiente
3. **Pedido: validación total > 0** (al menos producto o torta)
4. **Validación anticipo ≤ total del pedido**
5. **Pago final ≤ saldo pendiente** del pedido
6. **Movimiento de stock tipo Salida** — no puede exceder stock disponible
7. **Restricciones por rol** — vendedor no puede crear productos ni usuarios
8. **Integridad de claves foráneas** — producto, cliente, usuario deben existir

3. Requisitos que requieren validación

- Campos obligatorios en formularios (login, productos, clientes, usuarios, pedidos, stock, ventas)
- Rangos numéricos (precio ≥ 0 , cantidad ≥ 1 , anticipo ≥ 0 , pago ≥ 0.01)
- Longitudes máximas de campos de texto
- Formato y unicidad de carnet (clientes y usuarios)
- Unicidad de nombre de usuario
- Confirmación de contraseña al crear/editar usuario
- Tipos MIME y tamaño máximo (2MB) de imágenes de productos
- Longitud mínima de contraseña (6 caracteres)
- Fecha de entrega debe ser una fecha válida

4. Reglas de negocio que deben comprobarse

ID	Regla	Módulo
RN-01	Solo usuarios con estado=1 pueden iniciar sesión	Auth
RN-02	El anticipo no puede ser mayor que el total del pedido	Pedidos
RN-03	El pago final no puede ser mayor que el saldo pendiente	Pedidos
RN-04	Un pedido debe tener al menos un producto (cant >0) o una torta personalizada válida	Pedidos
RN-05	Las ventas directas descuentan stock automáticamente	Ventas
RN-06	No se permite venta si cantidad > stock disponible	Ventas
RN-07	Los pedidos NO descuentan stock (producto se elabora)	Pedidos
RN-08	Movimiento Salida: stock final no puede ser negativo	Stock
RN-09	Rol Administrador: acceso completo	Permisos
RN-10	Rol Vendedor: sin acceso a productos/create, stock/create, usuarios/*	Permisos

5. Riesgos potenciales

1. **Race condition en ventas:** Dos ventas concurrentes podrían vender más stock del disponible.
2. **Pérdida de integridad sin transacciones:** Si ocurre un error a mitad de la creación de pedido/venta, podrían quedar datos huérfanos.
3. **Acceso no autorizado por URL directa:** Usuarios vendedores podrían intentar acceder a rutas de administrador.
4. **Subida de archivos maliciosos** en imágenes de productos.
5. **Inconsistencia de saldo/anticipo:** Error de cálculo en actualización de pagos parciales.
6. **Usuario inactivo que sigue con sesión abierta** si se desactiva mientras está logueado.
7. **Stock negativo** por ajustes manuales incorrectos.

6. Datos que deberían utilizarse durante las pruebas

- **Roles predefinidos:** 1=Administrador, 2=Vendedor
- **Estados de pedido:** 1=Pendiente, 2=En preparación, 3=Listo, 4=Entregado, 5=Cancelado
- **Tipos de movimiento:** Entrada, Salida, Ajuste
- **Categorías de producto:** Al menos 2 categorías activas
- **Usuarios de prueba:** 1 admin activo, 1 vendedor activo, 1 usuario inactivo
- **Clientes:** 2-3 clientes (uno con carnet conocido para probar duplicados)
- **Productos:** 3-5 productos con stock variado (0, 3, 10, 100)
- **Pedidos de prueba:** Distintos estados y saldos

7. Tipos de pruebas recomendados

1. **Pruebas Unitarias:** Validaciones de modelos, cálculos de totales/saldos
2. **Pruebas Funcionales (Manuales):** Todos los flujos de usuario
3. **Pruebas de Integración:** Flujos completos (crear cliente → crear pedido → pagar → entregar)
4. **Pruebas de Autenticación/Autorización:** Acceso por rol y URLs protegidas
5. **Pruebas de Validación de Entrada:** Campos vacíos, límites, datos inválidos, duplicados
6. **Pruebas de Base de Datos/Persistencia:** CRUD, FK, integridad referencial
7. **Pruebas de Transacciones:** Verificar rollback en fallos
8. **Pruebas de Navegación y UX:** Enlaces, redirecciones, mensajes de error/éxito
9. **Pruebas de Rendimiento Básicas:** Carga del dashboard con datos voluminosos
10. **Pruebas de Regresión:** Después de cada cambio, re-ejecutar CP críticos

8. Dependencias entre módulos

```

``  Usuarios/Roles → Auth → Dashboard ↓ Productos → Stock → Ventas (descuenta
stock) ↓ ↑ Categorías Pedidos (no descuenta stock, usa productos) ↑ Clientes
_____ EstadosPedido _____ TiposMovimiento → Stock ``

```

9. Funcionalidades que podrían generar errores

1. Pedido con cero productos y torta incompleta
2. Pago final que excede saldo
3. Movimiento de Salida con stock 0
4. Venta de producto sin stock
5. Usuario intenta acceder con credenciales correctas pero cuenta desactivada
6. Edición de usuario con mismo carnet/usuario de otro registro
7. Subida de imagen de producto > 2MB

8. Anticipo > total del pedido
9. Pedido con productos y cantidades = 0 en todos
10. Acceso directo a URL /usuarios con rol Vendedor

10. Información que necesito proporcionar para elaborar el plan de pruebas correctamente

? Faltante (requiere confirmación):

1. Estructura exacta de migraciones/tablas MySQL (actualmente la migración users no coincide con tabla usuarios)
2. IDs y nombres exactos de catálogos (roles, estados_pedido, tipos_movimiento)
3. Requisitos no funcionales explícitos (tiempos, concurrentes, disponibilidad)
4. Formato exacto de carnet y teléfono
5. Reglas de negocio adicionales (editar/eliminar clientes, soft delete, historial)
6. País/moneda confirmación
7. Criterios de aprobación (defectos permitidos, pase a producción)
8. Flujo de estados (saltos permitidos, cancelación, reversión)
9. Credenciales iniciales del administrador
10. Búsqueda/filtrado, paginación, reportes